Banking Management System

Using Arrays, Linked Lists, and Qt GUI

Developed By:

Med Naceur Chouchane

Med Hedi Ben Rhouma

Zied Abdennour

Ramzi Fredj

Course/Subject: Data Structures and Algorithms

Date: December 2025

Framework: Qt 6.10.1 with C++17

Table of Contents

1. Introduction

This project presents a comprehensive Banking Management System designed to efficiently manage customer accounts, employee records, loan processing, and financial transactions. The system implements core data structures including **arrays** for fixed-size collections, **doubly linked lists** for dynamic loan management, and **singly linked lists** for archived data storage.

Key algorithms implemented include **linear search** for customer/employee lookup, **sorting algorithms** for data organization, and **traversal operations** for linked list manipulation. The application utilizes **C++** as the primary programming language, **Qt Designer** for professional GUI design, and **CMake** as the build system.

The system achieves a fully functional GUI-based banking solution with **CSV file persistence**, supporting operations for three user roles (customers, employees, managers) with over 25 interconnected windows and real-time data validation.

Project Scope

Customer Operations

- Account registration
- Deposit/Withdrawal
- Loan requests
- Transaction history

Employee Operations

- Add customers
- Manage accounts

- Process loans
- Daily operations

Manager Operations

- Hire/Fire employees
- View statistics
- Access archives
- Generate reports

Teamwork & Collaboration

An essential aspect of this project was the strong teamwork maintained throughout its development.

Our group strategically split into two teams working simultaneously on different components of the system: one team focused on data structure implementation and backend logic, while the other worked on the Qt GUI design and user interaction flow.

Although coordinating parallel development introduced challenges—such as merging features, aligning coding styles, and resolving unexpected integration issues—we consistently communicated, reviewed each other's work, and adapted when necessary.

These collaborative efforts allowed us to overcome obstacles efficiently and ultimately deliver a unified, fully functional banking management system.

This experience strengthened our technical abilities as well as our capacity to work as an effective software development team.

2. Problem Statement

2.1 Challenges in Manual Banking Management

1. **Error-Prone Manual Processing:** Manual record-keeping leads to human errors in calculations, data entry, and record retrieval, potentially causing financial discrepancies.
2. **Inefficient Data Retrieval:** Searching through physical records or poorly organized digital files wastes time and reduces productivity.
3. **Limited Scalability:** As customer base grows, manual systems become increasingly difficult to manage without proper data structure implementation.
4. **Data Integrity Issues:** Without proper validation and concurrent access control, data corruption and inconsistencies arise.

2.2 Project Solution

This system addresses these challenges through strategic data structure selection and algorithm implementation:

Data Structure Selection

Arrays: Fixed capacity (100 customers/employees) provides O(1) access time with predictable memory usage, suitable for controlled-scale applications.

Linked Lists: Dynamic loan storage allows unlimited loan history per customer with efficient insertion/deletion operations.

3. Methodology

3.1 Data Structures Implementation

3.1.1 Static Arrays

> **Structure:** `customer cust[100]` , `employee employees[100]`
>
> **Complexity:** O(1) access, O(n) search, O(n) insertion/deletion
>
> **Justification:** Predictable memory allocation, cache-friendly access patterns

3.1.2 Doubly Linked List (Loans)

> **Structure:** `struct Node { loan data; Node* next; Node* previous; }`
>
> **Complexity:** O(1) insertion at ends, O(n) search, O(1) deletion with pointer
>
> **Justification:** Supports unlimited loans per customer, enables bidirectional traversal

3.2 Algorithms Applied

> **Search Algorithms**
>
> - **Linear Search:** Customer lookup by account number O(n)
> - **Sequential Scan:** Employee ID verification

Sorting Algorithms

- **Bubble Sort:** Employee sorting by name/branch
- **Date Comparison:** Sorting by hire date

6. System Screenshots & Results

Figure 0: *Main Banking Systeme interface*

6.1 User Registration & Authentication

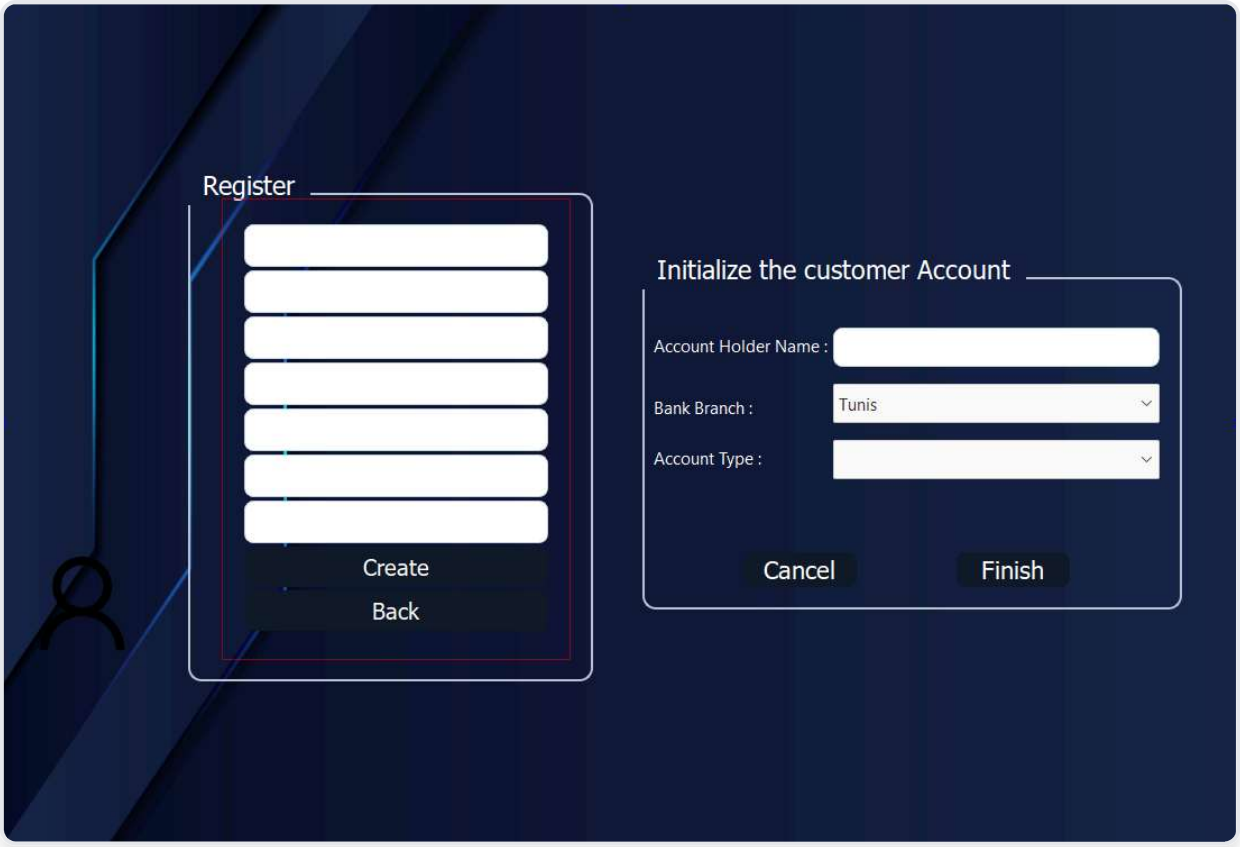

Figure 1: *Customer registration interface showing multi-field form for personal information entry with validation.*

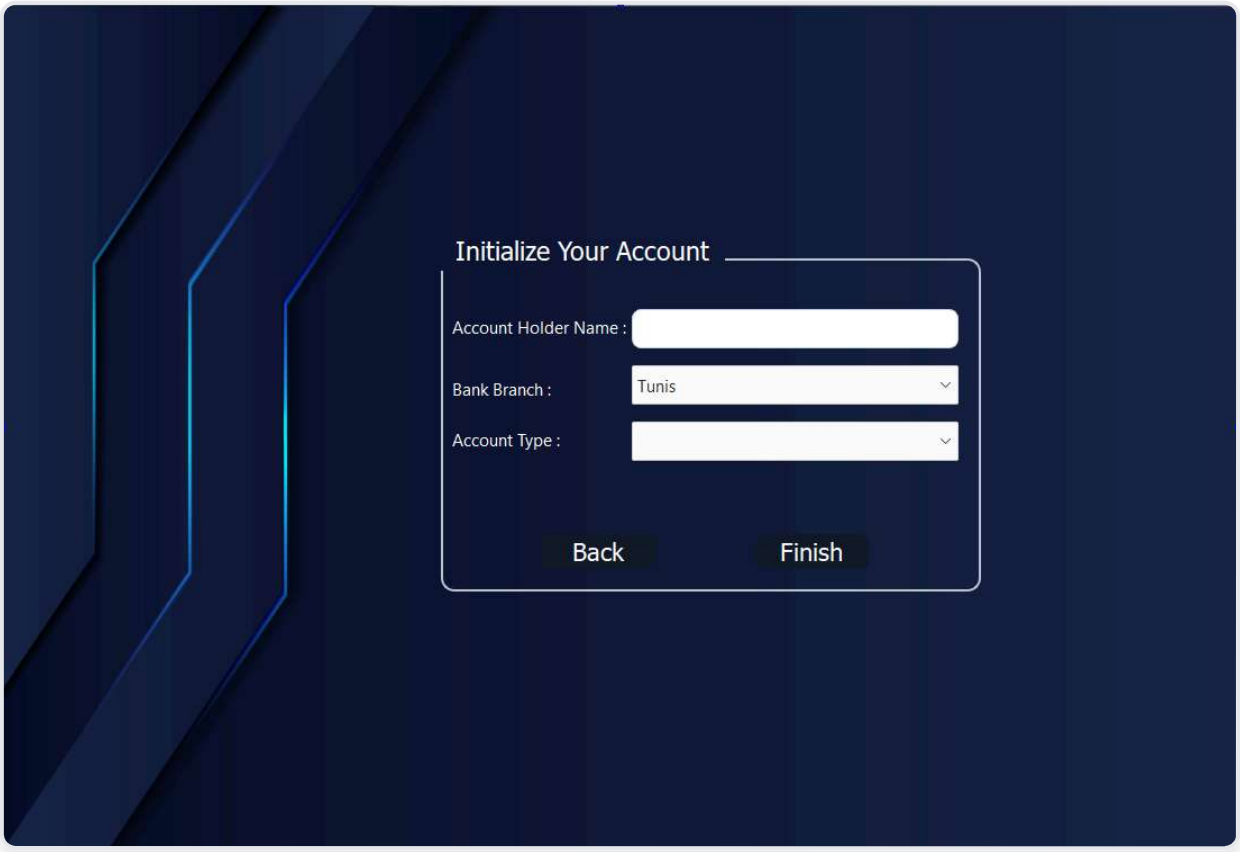

Figure 2: *Account initialization dialog with bank branch and account type selection, featuring auto-generated IBAN.*

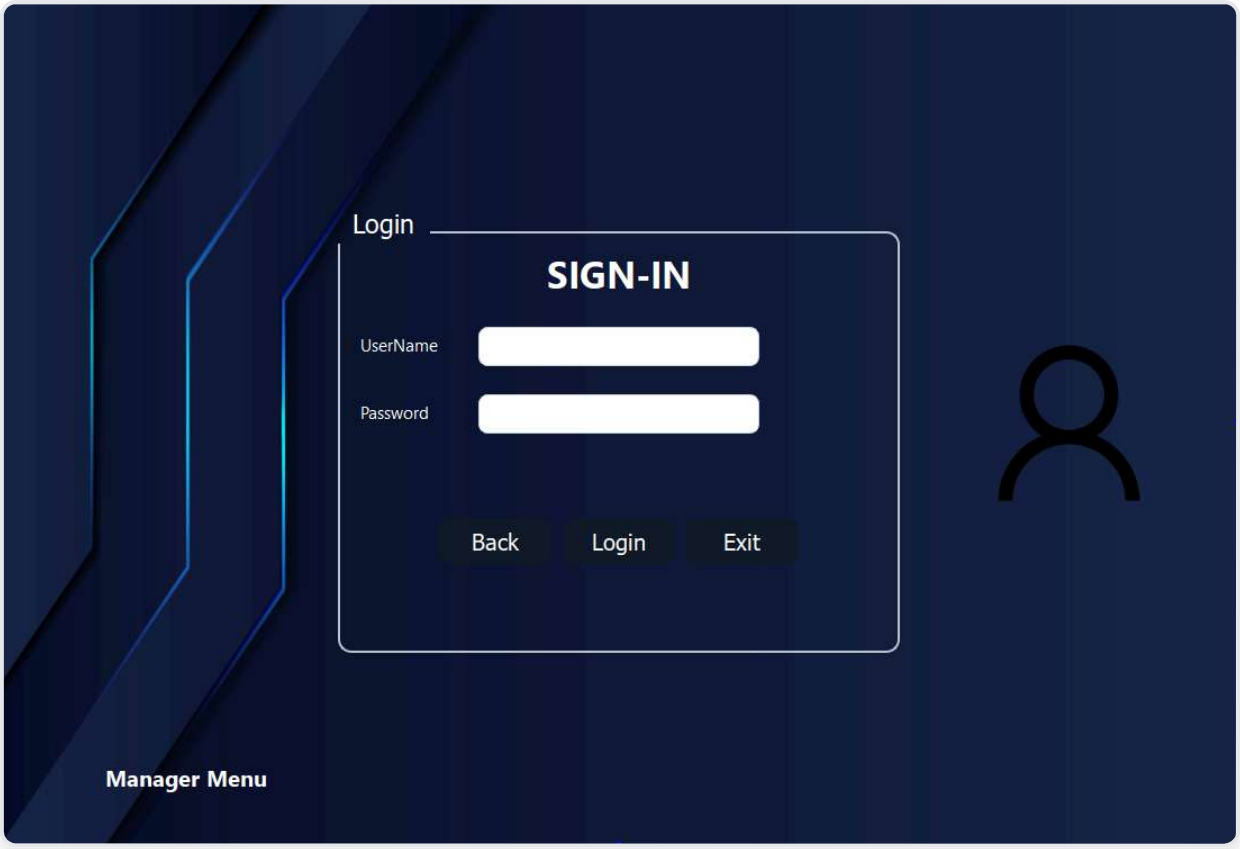

Figure 3: *Unified login interface (SIGN-IN) for customer, employee, and manager authentication using linear search algorithm.*

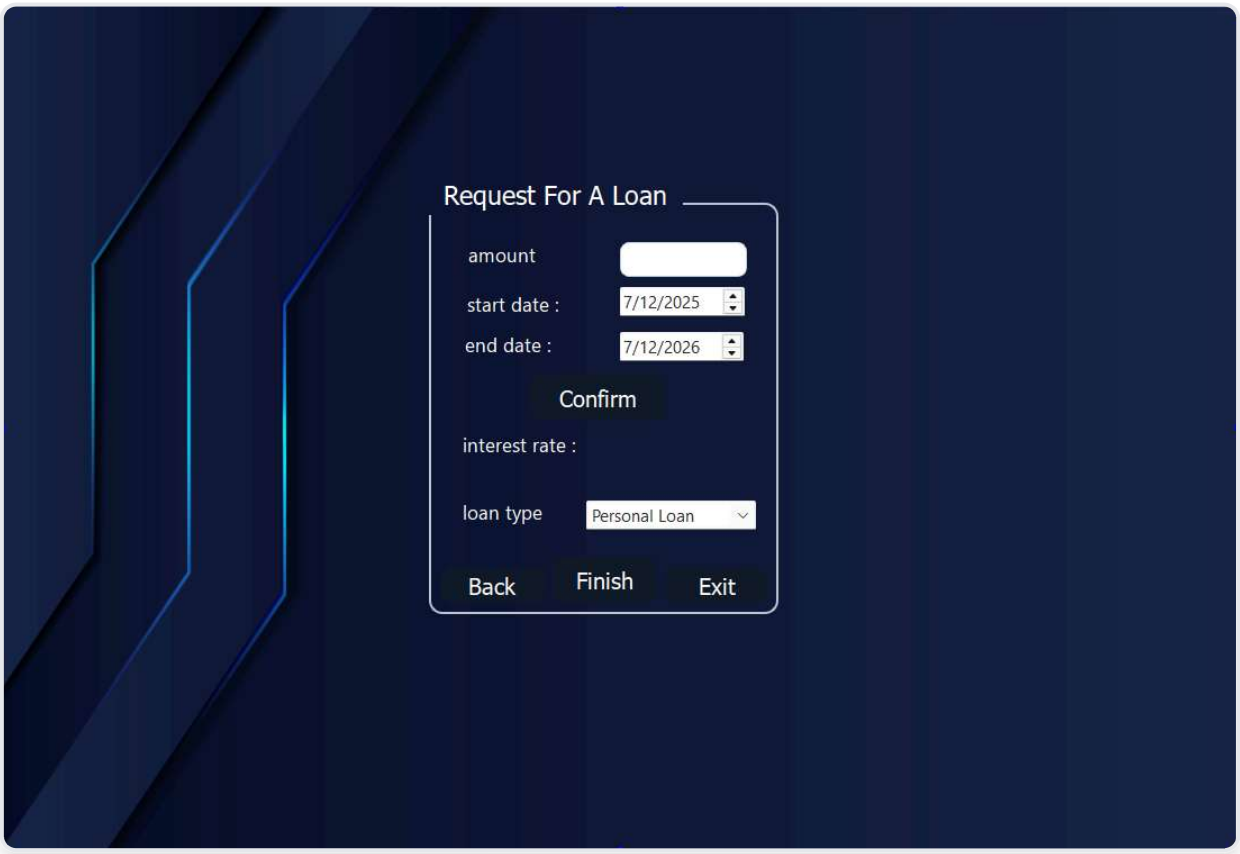

Figure 4: *A loan Request interface*

6.2 Customer Portal

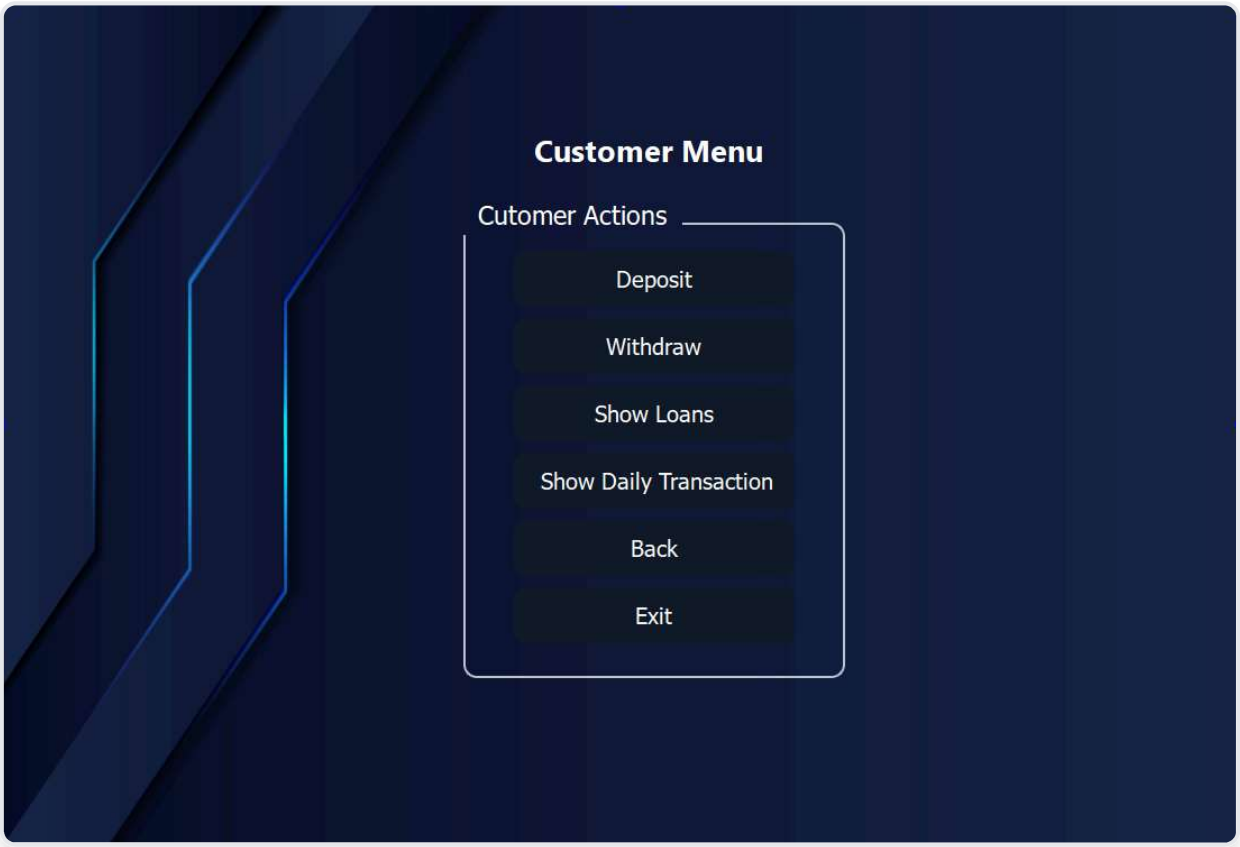

Figure 5: *Customer dashboard displaying available banking operations: Deposit, Withdraw, Show Loans, Show Daily Transaction.*

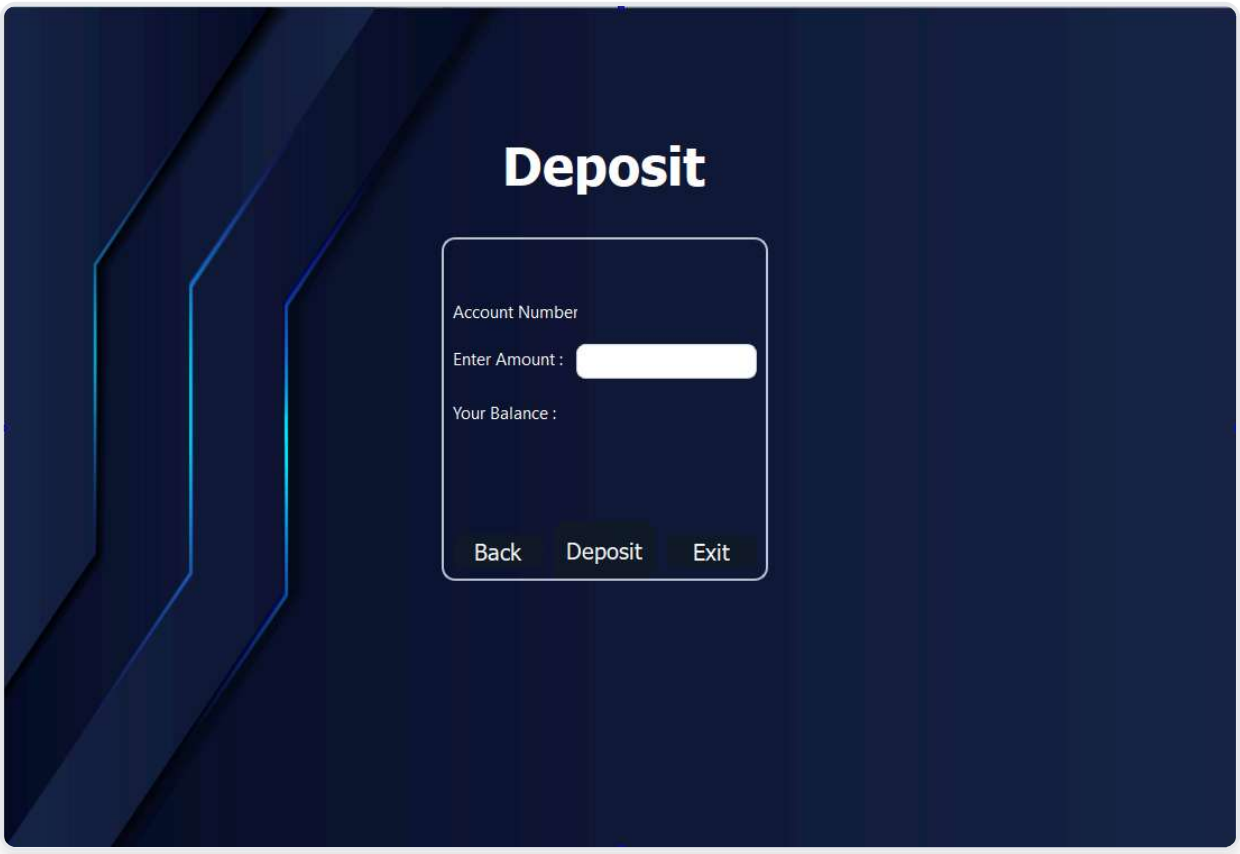

Figure 6: *Deposit transaction interface with amount input, current balance display, and real-time balance updates.*

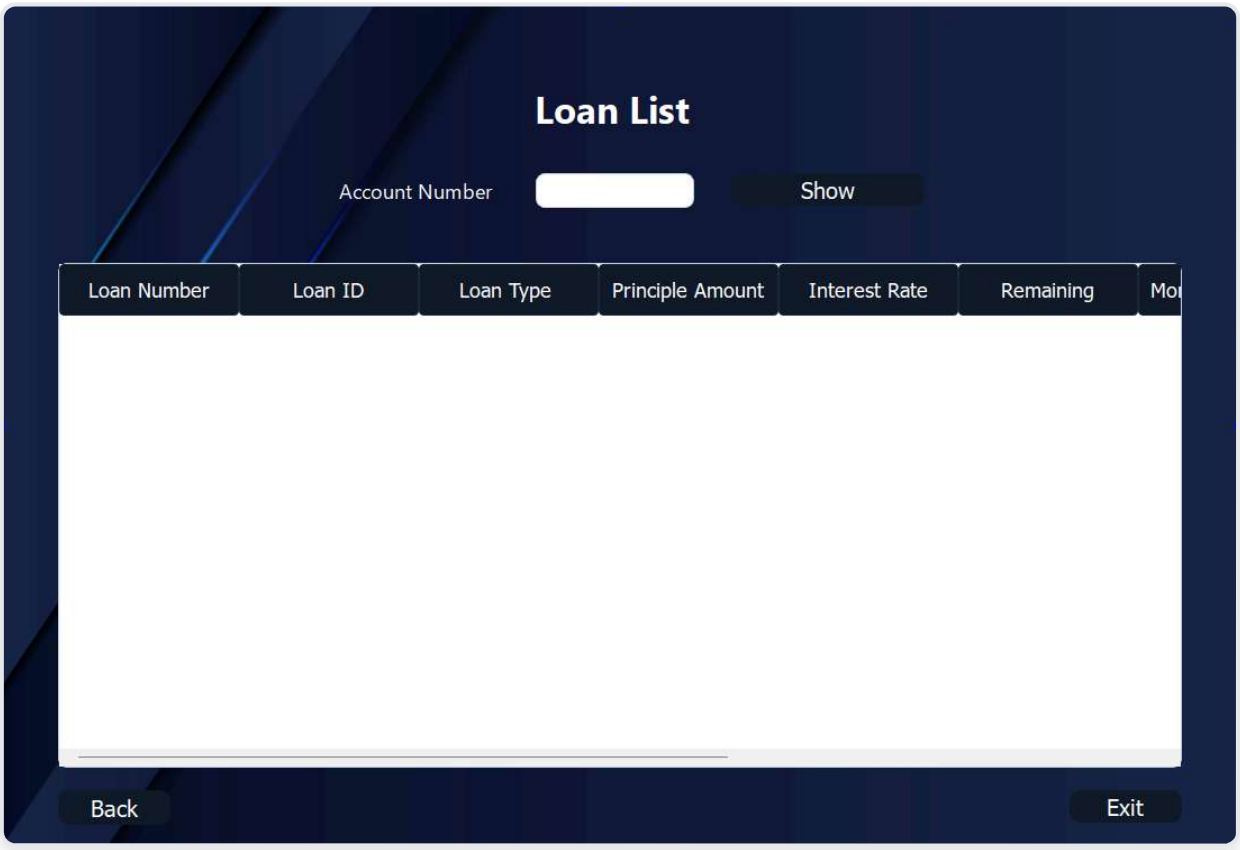

Figure 7: *Loan display using doubly linked list traversal, showing Loan Number, ID, Type, Principal, Interest Rate, and Monthly Payment.*

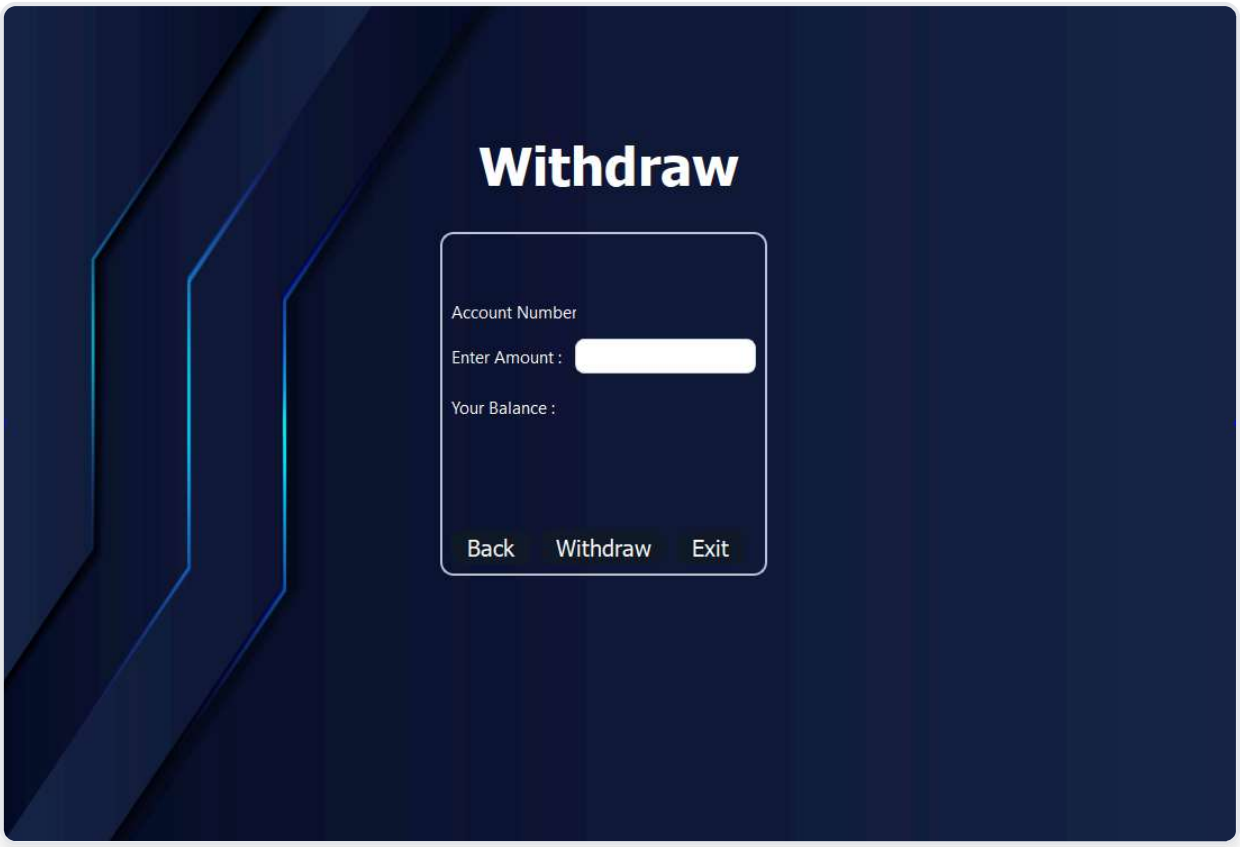

Figure 8: *Withdraw Money and display remaining balance.*

6.3 Transaction Management

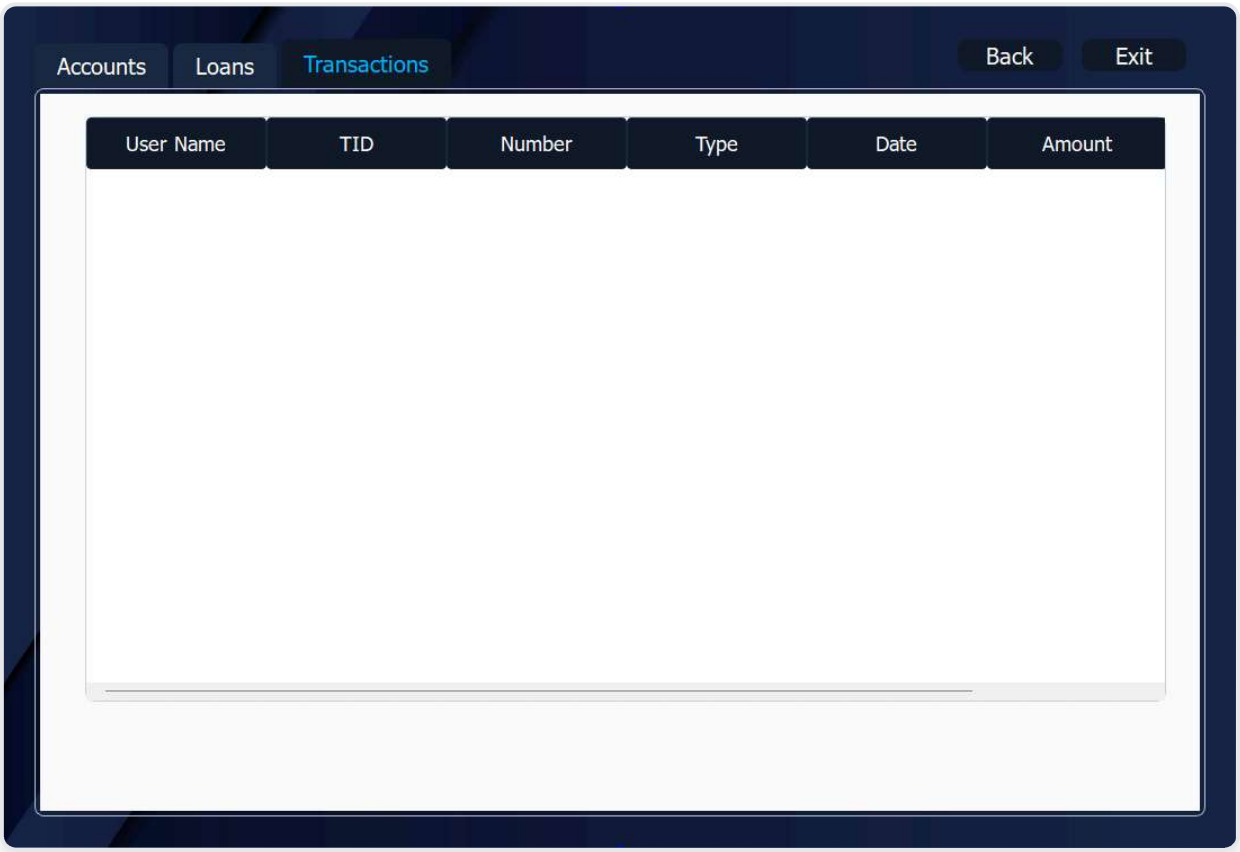

Figure 7: *Daily transaction history with tabbed interface (Accounts/Loans/Transactions) displaying chronological transaction records.*

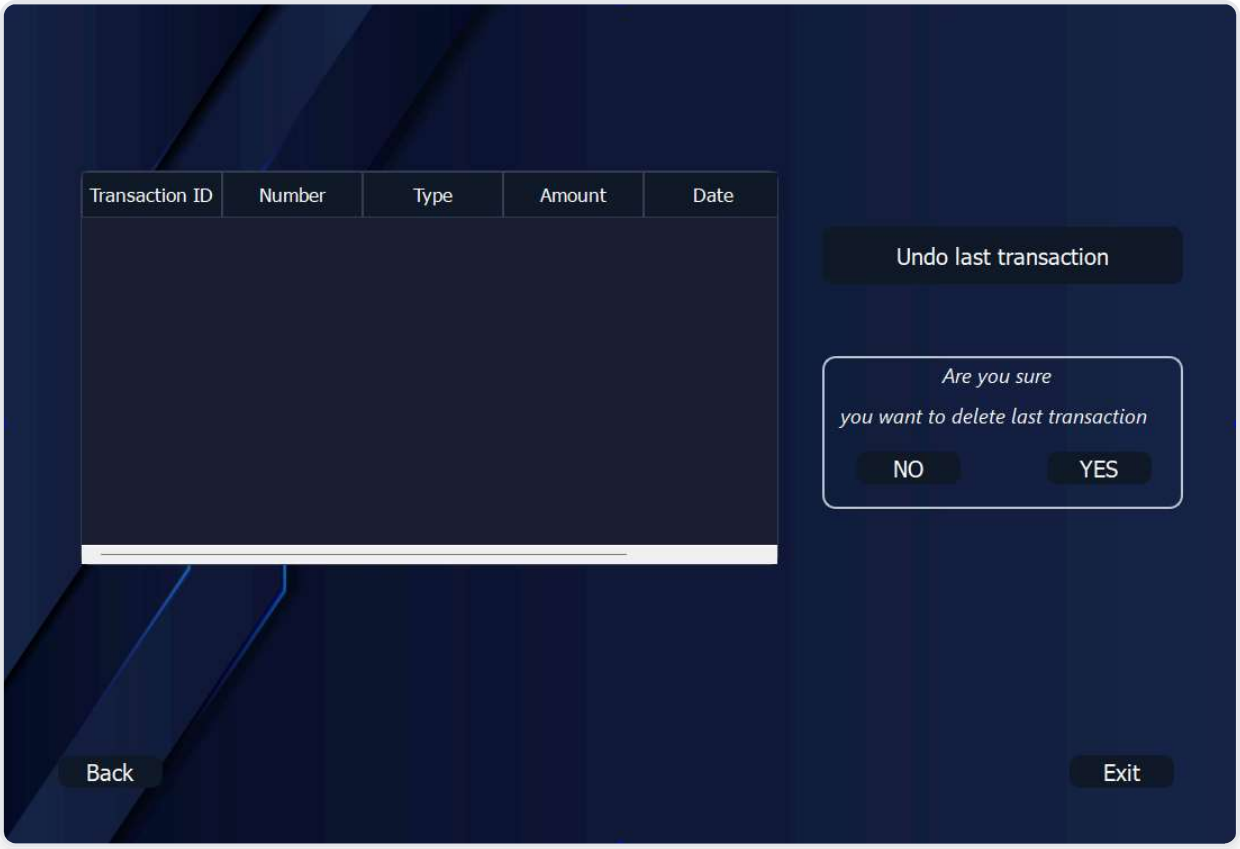

Figure 9: *Undo last transaction feature with confirmation dialog, implementing LIFO stack operation for transaction reversal.*

6.4 Employee Operations

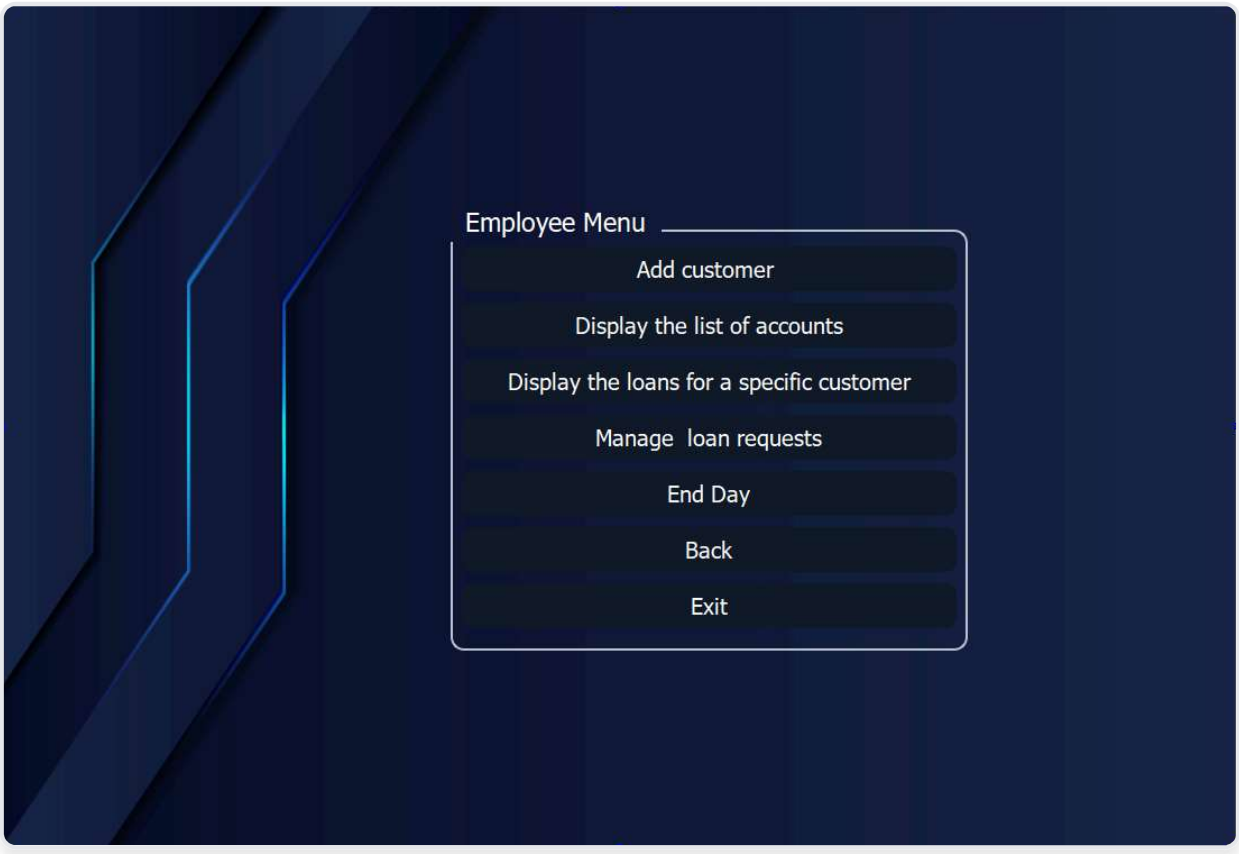

Figure 10: *Employee dashboard with access to customer management, account operations, loan processing, and daily reporting.*

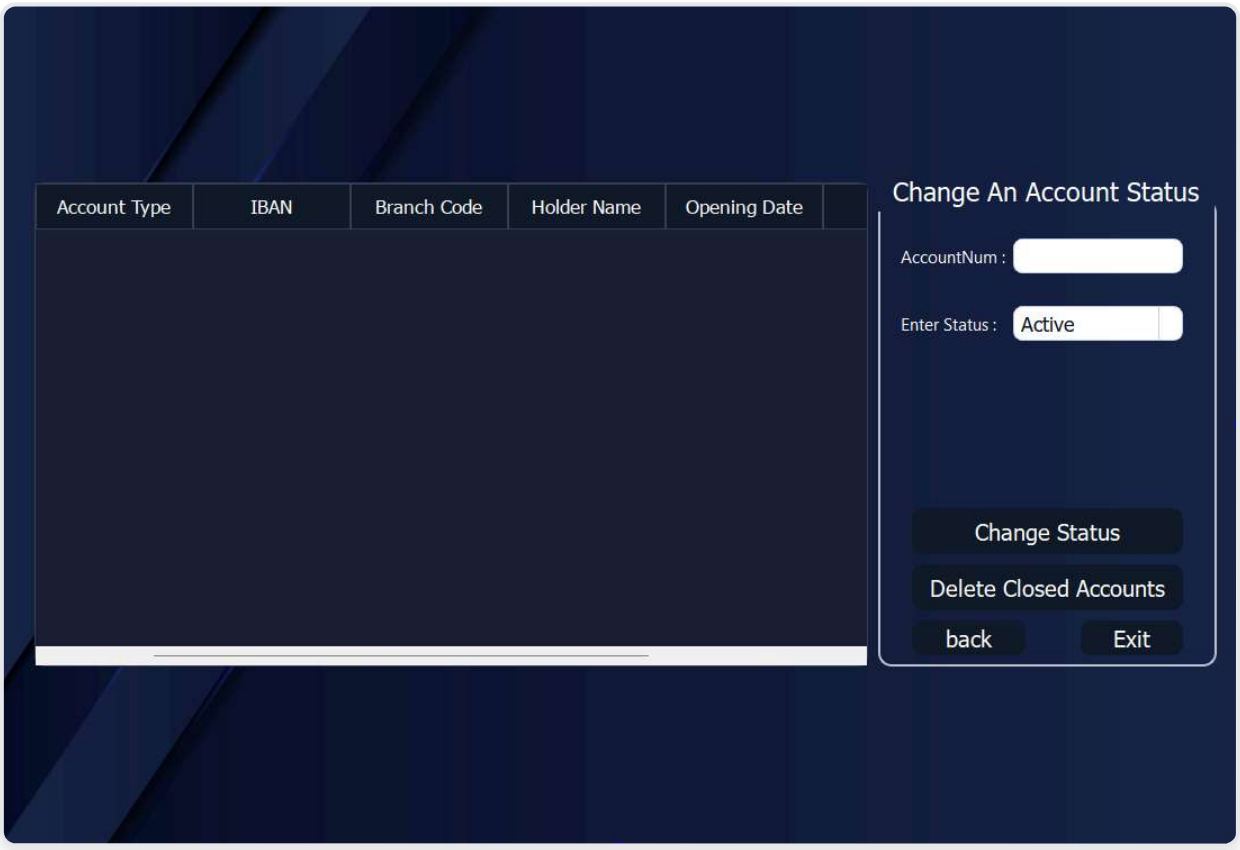

Figure 11: *Account status management interface allowing employees to change account states and delete closed accounts.*

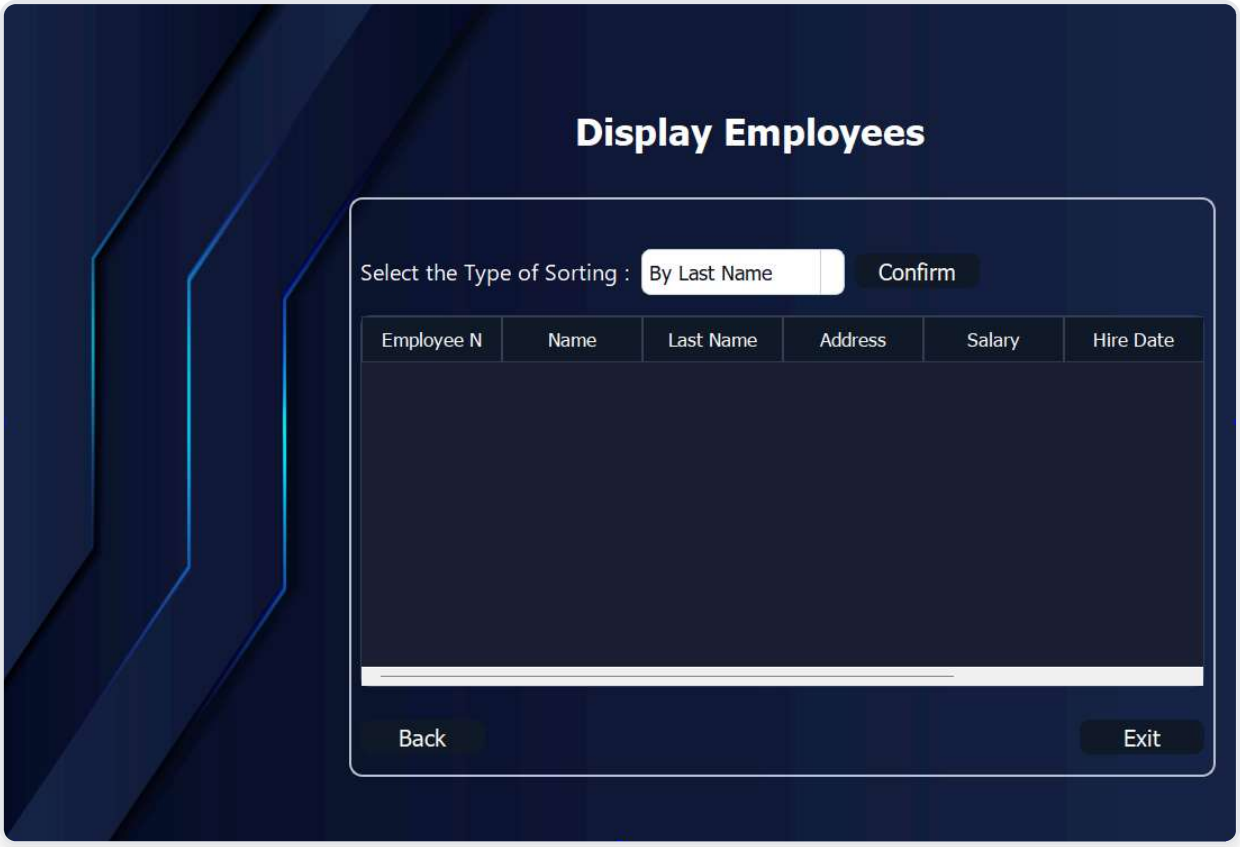

Figure 12: *Employee display with sorting functionality (by last name shown), implementing bubble sort algorithm.*

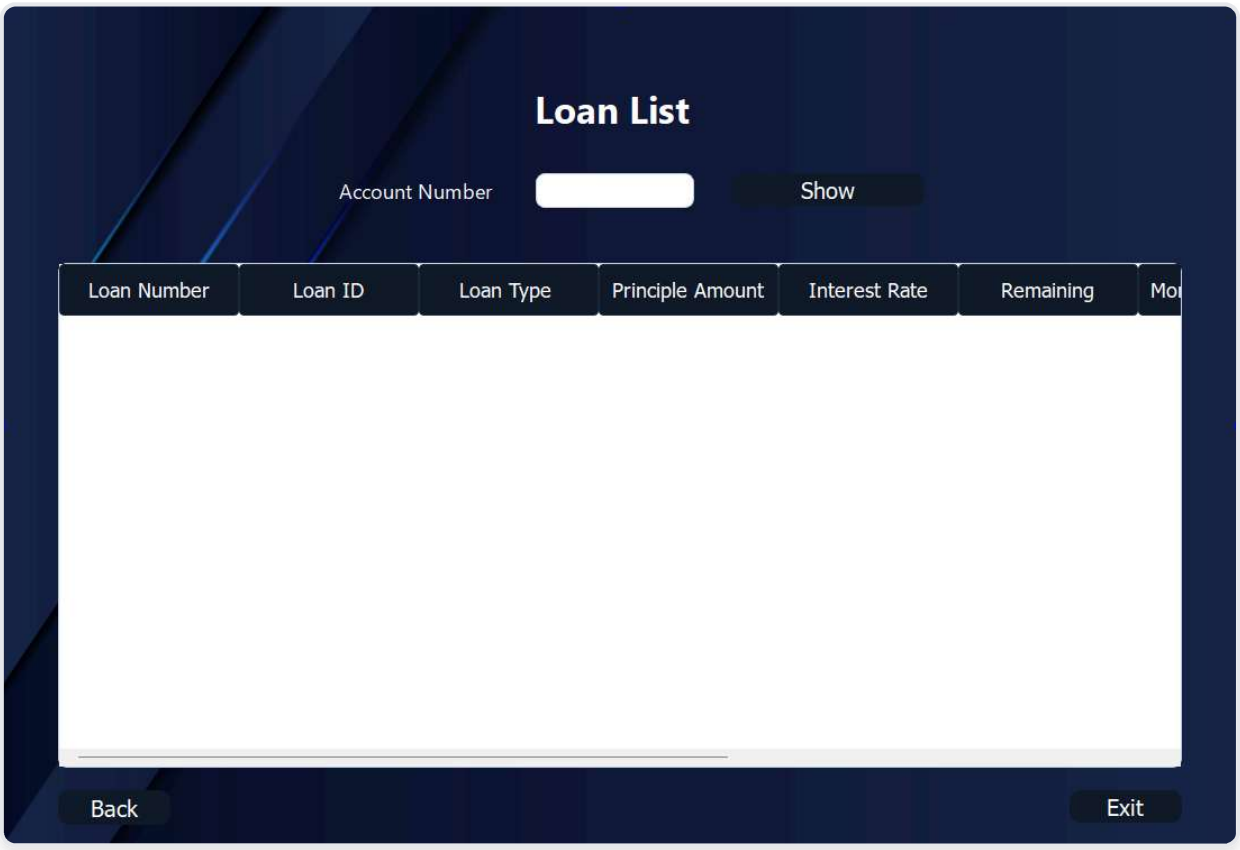

Figure 13: *Employee loan lookup by account number, displaying linked list of all customer loans with financial details.*

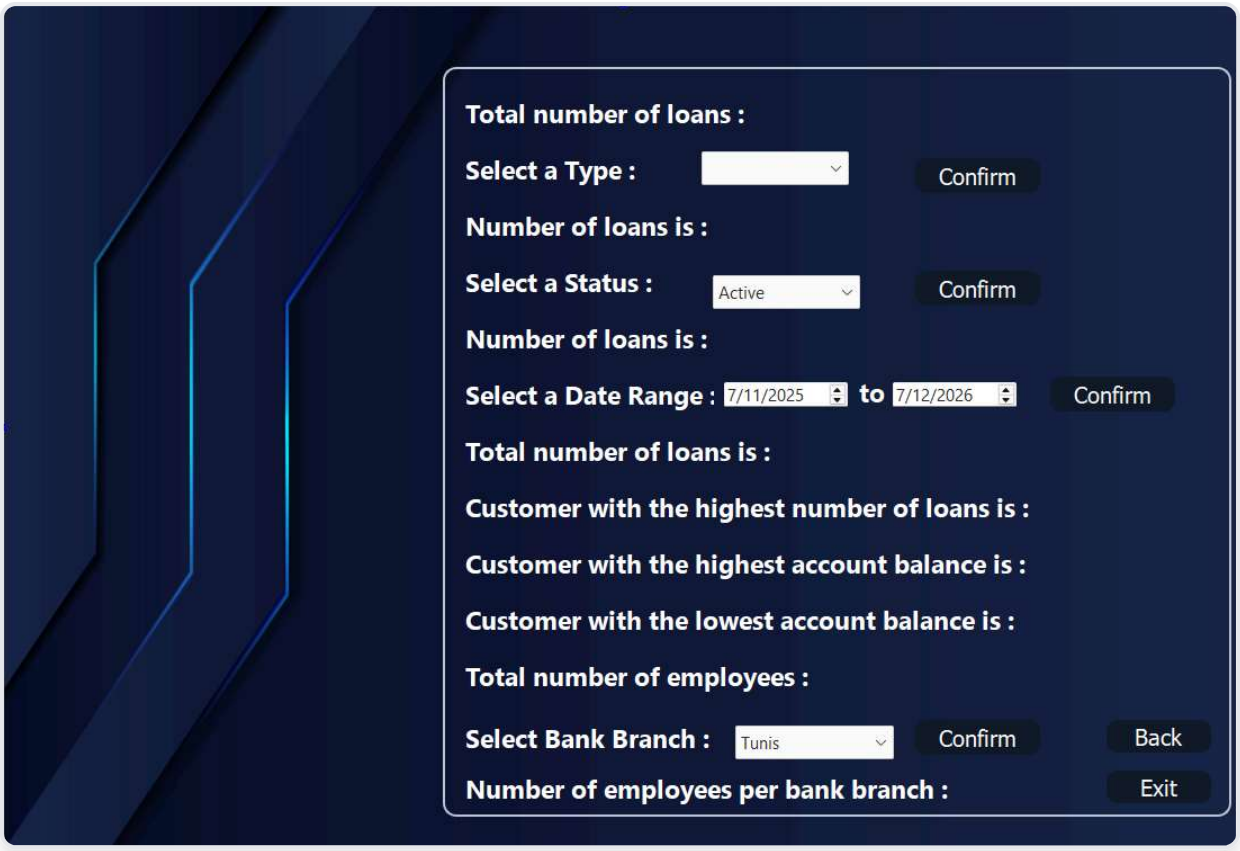

Figure 14: *Modification of any specific detail rellated to a customer*

6.5 Manager Operations

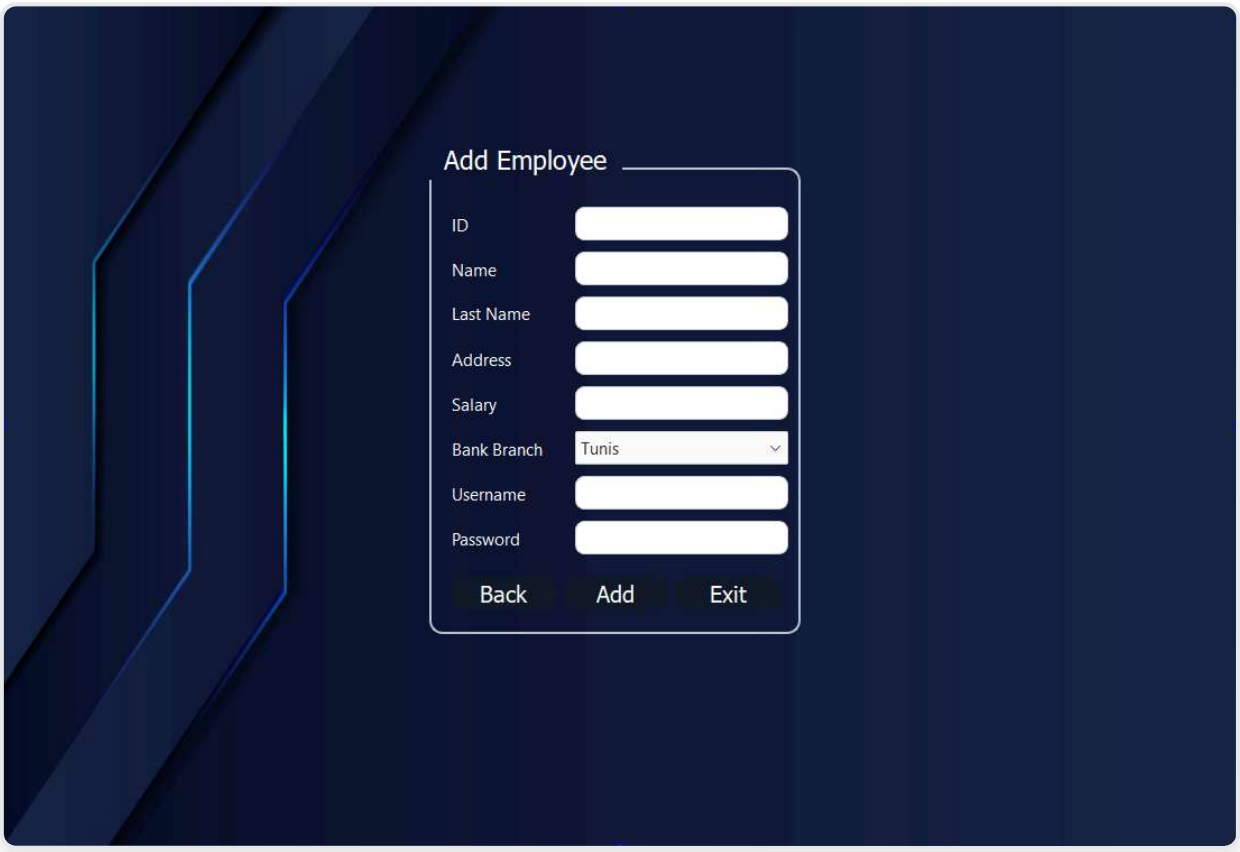

Figure 15: *Employee hiring interface for managers with comprehensive input validation before adding to employee array.*

Figure 16: *Manager statistics dashboard with loan analytics, customer metrics, and employee distribution by branch.*

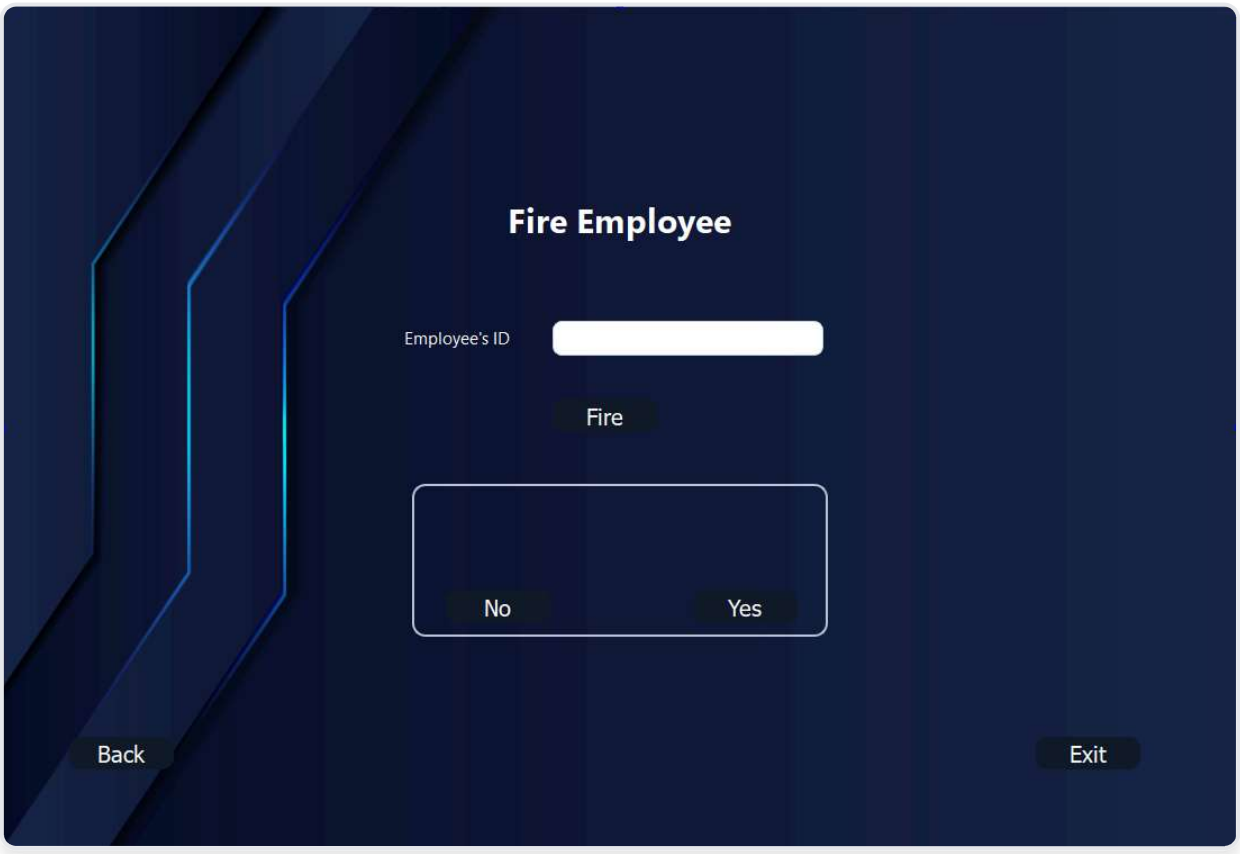

Figure 17: *Firing employee possibility*

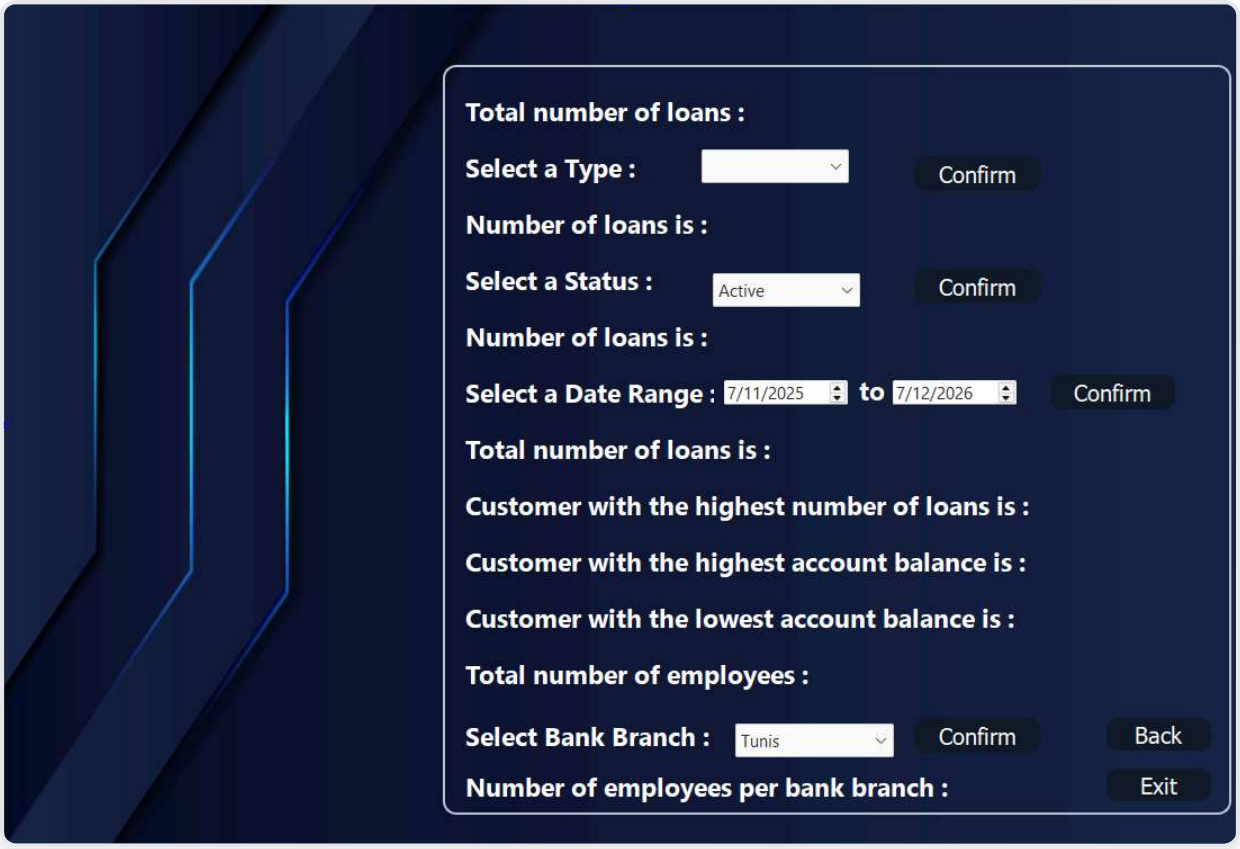

Figure 18: *Statistics interface to display Streamlined loan and employee insights at your fingertips—track, filter, and analyze with ease.*

6.6 Key Features Demonstrated

Data Structure Usage

- **Arrays:** Customer/Employee storage (O(1) access)
- **Doubly Linked Lists:** Loan history (bidirectional traversal)
- **Transaction Arrays:** Financial history tracking

Algorithm Implementation

- **Linear Search:** Login authentication, account lookup
- **Bubble Sort:** Employee sorting by multiple fields
- **Linked List Traversal:** Loan display and statistics

7. Challenges & Solutions

Challenge 1: Nested Data Structure Complexity

> **Problem:** Managing customer → account → loans (linked list) + transactions (array) created type confusion and pointer errors.
>
> **Solution:** Clearly defined separate header files for each structure level with consistent naming conventions and helper functions.

Challenge 2: Qt Integration & MOC Compilation

> **Problem:** Meta-Object Compiler (MOC) errors when Qt classes weren't properly declared.
>
> **Solution:** Ensured all Qt classes include Q_OBJECT macro and properly configured CMakeLists.txt with CMAKE_AUTOMOC.

Challenge 3: CSV Data Corruption During Updates

> **Problem:** Direct file modification could corrupt data if program crashed mid-write.
>
> **Solution:** Implemented atomic file replacement strategy: write to temporary file → close both → delete original → rename temp.

Challenge 4: Memory Leaks in Linked Lists

Problem: Deleting customers/employees without properly freeing their loan linked lists caused memory leaks.

Solution: Implemented destroyList() function to recursively delete all nodes with proper cleanup in destructors.

8. Conclusion

8.1 Project Achievements

- **Complete Functional Implementation:** All planned features successfully implemented and tested
- **Effective Data Structure Usage:** Strategic deployment of arrays and linked lists for optimal performance
- **Professional GUI Development:** Intuitive interface comparable to commercial banking applications
- **Robust Data Persistence:** CSV-based storage with atomic update mechanisms ensuring data integrity
- **Multi-Role Access Control:** Successfully implemented three distinct user roles with appropriate permissions

8.2 Learning Outcomes

> **Technical Skills Acquired**
>
> - Deep understanding of static arrays vs. dynamic structures trade-offs
> - Practical experience with doubly and singly linked list implementations
> - Proficiency in Qt framework for cross-platform GUI development
> - File I/O operations and data serialization techniques
> - Algorithm complexity analysis (Big-O notation) in real contexts
> - Memory management and pointer manipulation in C++

8.3 Future Improvements

Short-term Enhancements

- Implement binary search trees for O(log n) lookups
- Add hash tables for O(1) average-case searches
- Integrate SQLite database for better persistence
- Implement password hashing (SHA-256)
- Add data export to Excel/PDF formats

Long-term Vision

- Client-server architecture with Qt networking
- PostgreSQL/MySQL integration for enterprise scale
- Advanced analytics with chart libraries
- Email notifications for loan approvals
- Mobile application companion (Qt for Android/iOS)
- RESTful API for third-party integration

8.4 Final Remarks

The development of this Banking Management System provided invaluable hands-on experience in applying theoretical computer science concepts to practical software development. The challenges encountered—from Qt integration issues to CSV data corruption handling—taught us resilience and systematic debugging approaches.

Most importantly, this project demonstrated that choosing the right data structure is as crucial as writing correct code. The decision to use arrays for primary storage traded scalability for simplicity and performance, while linked lists enabled flexible loan management without predefined size limits.

> **Key Takeaway:** Data Structures and Algorithms are not merely academic concepts but fundamental tools that directly impact software performance, maintainability, and user experience.

9. References & Resources

9.1 AI-Assisted Development Tools

1. **Claude AI (Anthropic)** - Used for debugging complex Qt MOC compilation errors, explaining parent-child widget communication patterns, and resolving memory leak issues in linked list implementations.
   URL: https://claude.ai
2. **Google Gemini** - Consulted for clarifying C++ pointer semantics, validating data structure design decisions, and assistance with CSV parsing edge cases.
   URL: https://gemini.google.com

9.2 Video Tutorials & Learning Resources

3. **Eng Ahmed Sameh YouTube Channel** - Comprehensive Qt tutorials covering:
   - Qt Designer UI file integration with C++ backend
   - Signal and slot connections between widgets
   - QTableWidget population and manipulation
   - Parent-child widget hierarchy management
   - Qt application structure and best practices

   Channel: Eng Ahmed Sameh (YouTube)

9.3 Official Documentation

4. **Qt Documentation (Qt 6.10)** - Official reference for Qt classes and widgets
   URL: https://doc.qt.io/qt-6/
5. **C++ Reference** - Comprehensive C++ language reference
   URL: https://en.cppreference.com/
6. **CMake Documentation** - Build system configuration
   URL: https://cmake.org/documentation/

9.4 Academic Resources

7. **Data Structures and Algorithms in C++** Class courses :The usage of singly and doubly linked list ,dynamic arrays,stacks and data structure logic

---

Banking Management System - Project Report

Developed by: Med Naceur Chouchane, Med Hedi Ben Rhouma, Zied Abdennour, Ramzi Fredj

Data Structures and Algorithms Project | December 2025